

Université — Département Informatique

Module : Optimisation Combinatoire

Étude Comparative de Méthodes de Résolution de l'Orienteering Problem

Problème : Orienteering Problem (OP)

Approches : 3 exactes + 3 métaheuristiques

Résumé

Ce rapport présente une étude comparative de six méthodes d'optimisation appliquées à l'*Orienteering Problem* (OP), un problème NP-difficile combinant sélection de sommets et ordonnancement de tournée sous contrainte de budget de distance. Trois méthodes exactes sont implémentées : Programmation Dynamique par masque de bits, Branch & Bound et Programmation Linéaire Entière (ILP) : ainsi que trois métaheuristiques : Recuit Simulé, Recherche Tabou et GRASP. Chaque approche est analysée théoriquement puis évaluée expérimentalement sur des instances de tailles croissantes. Les résultats montrent que la Recherche Tabou offre le meilleur compromis qualité/temps sur les instances de taille moyenne à grande, tandis que la DP reste très rapide pour les petites instances ($n \leq 22$).

Table des matières

1	Introduction	2
1.1	Présentation du problème	2
1.2	Formulation mathématique	2
1.3	Complexité et enjeux	2
1.4	Objectif du projet	3
2	Guide de Démarrage Rapide	3
3	Architecture du Projet	4
3.1	Structure des modules	4
3.2	Structures de données principales	4
3.3	Génération des instances	5
4	Méthodes Exactes	5
4.1	Programmation Dynamique par masque de bits (DP)	5
4.2	Branch & Bound (B&B)	6
4.3	Programmation Linéaire Entière (ILP)	7
5	Métaheuristiques	7
5.1	Recuit Simulé	7
5.2	Recherche Tabou	8
5.3	GRASP	9
6	Évaluation Expérimentale	10
6.1	Protocole	10
6.2	Résultats par taille d'instance	10
6.3	Analyse de la complexité en temps	11
7	Discussion	11
7.1	Comparaison des approches	11
7.2	Limites et perspectives	12
8	Conclusion	12

1 Introduction

1.1 Présentation du problème

L'**Orienteering Problem** (OP) est un problème d'optimisation combinatoire introduit par GOLDEN ET AL. (1987) dans le contexte de la planification d'itinéraires touristiques. Il généralise le *Traveling Salesman Problem* (TSP) en ajoutant une dimension de sélection : il n'est pas nécessaire de visiter tous les sommets.

Formulation informelle. Étant donné un graphe pondéré de n sommets, un sommet de départ s , un sommet d'arrivée e , un score (profit) p_i associé à chaque sommet i , et un budget maximal de distance $T_{\max}$, l'objectif est de trouver une tournée partant de s , se terminant en e , respectant $T_{\max}$, et maximisant la somme des scores des sommets visités.

1.2 Formulation mathématique

Soit $G = (V, E)$ un graphe complet avec $V = \{0, 1, \dots, n-1\}$. Posons $s = 0$ (départ) et $e = n-1$ (arrivée). On définit :

- $x_{ij} \in \{0, 1\}$: vaut 1 si l'arc (i, j) est emprunté,
- $y_i \in \{0, 1\}$: vaut 1 si le sommet i est visité,
- d_{ij} : distance entre les sommets i et j ,
- p_i : profit du sommet i .

$$\max \sum_{i \in V} p_i y_i \tag{1}$$

$$\text{s.c.} \quad \sum_{j \neq s} x_{sj} = 1, \quad \sum_{i \neq e} x_{ie} = 1 \tag{2}$$

$$\sum_j x_{ji} = \sum_j x_{ij} = y_i, \quad \forall i \notin \{s, e\} \tag{3}$$

$$\sum_{(i,j) \in E} d_{ij} x_{ij} \leq T_{\max} \tag{4}$$

$$\text{Sous-tours éliminés (contraintes MTZ)} \tag{5}$$

$$x_{ij}, y_i \in \{0, 1\} \tag{6}$$

1.3 Complexité et enjeux

L'OP est **NP-difficile** par réduction depuis le TSP. Il n'existe pas d'algorithme polynomial garantissant l'optimalité pour le cas général, ni de schéma d'approximation

à ratio constant (FPTAS). La difficulté réside dans le couplage entre décision de visite et ordonnancement, ce qui justifie l'étude comparative de plusieurs approches exactes et heuristiques.

1.4 Objectif du projet

Ce projet a pour objectifs de :

1. implémenter six méthodes de résolution distinctes,
2. analyser leur complexité théorique,
3. comparer leurs performances empiriques sur des instances de tailles variées (8 à 60 clients),
4. formuler des recommandations pratiques.

2 Guide de Démarrage Rapide

Prérequis

- Python 3.7 ou supérieur
- Aucune installation manuelle requise (PuLP est installé automatiquement à la première exécution de la méthode ILP)

Lancer le projet

1. Dashboard web interactif (recommandé)

Ouvrir un terminal dans le dossier du projet et exécuter :

```
1 python main.py
```

Le dashboard s'ouvre automatiquement dans le navigateur par défaut. Il contient une interface complète avec six onglets : Aperçu, Méthodes, Démo, Comparaison, Exécuter et Documentation.

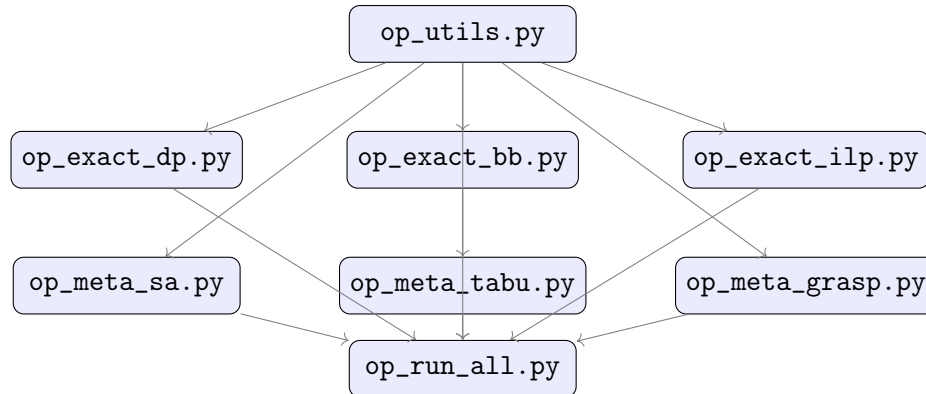
2. Exécuter une méthode spécifique

```
1 python main.py --method dp          % Programmation Dynamique
2 python main.py --method bb          % Branch & Bound
3 python main.py --method ilp         % ILP (PuLP/CBC)
4 python main.py --method sa          % Recuit Simule
5 python main.py --method tabu        % Recherche Tabou
6 python main.py --method grasp       % GRASP
7 python main.py --method tabu --n 30 % Tabu sur 30 clients
```

3 Architecture du Projet

3.1 Structure des modules

Le projet est organisé en trois couches :



3.2 Structures de données principales

Deux *dataclasses* Python centralisent toutes les données :

```

1 @dataclass
2 class OPInstance:
3     n      : int          # nombre total de sommets
4     start  : int          # indice du depart (= 0)
5     end    : int          # indice de l'arrivee (= n-1)
6     customers: List[int]  # indices clients (1 .. n-2)
7     dist    : np.ndarray   # matrice n x n des distances
8     scores  : np.ndarray   # profits par sommet
9     t_max   : float        # budget maximal
10    coords  : np.ndarray   # coordonnees 2D des sommets
  
```

Listing 1 – Classe OPInstance (op_utils.py)

```

1 @dataclass
2 class OPResult:
3     method      : str      # nom de la methode
4     tour         : List[int] # tournée obtenue
5     total_score  : float    # somme des profits
6     total_dist   : float    # distance totale
7     cpu_time     : float    # temps CPU (secondes)
8     optimal      : bool     # solution certifiée optimale ?
9     gap          : float    # écart à l'optimum (%)
10    extra        : dict     # données spécifiques à la méthode
  
```

Listing 2 – Classe OPResult (op_utils.py)

3.3 Génération des instances

Les instances sont générées aléatoirement dans un domaine carré $[0, 100]^2$:

- les départ et arrivée sont placés au centre,
- les profits $p_i \in \{1, \dots, 10\}$ sont tirés uniformément,
- $T_{\max} = \text{ratio} \times \max_{i,j} d_{ij}$ (par défaut $\text{ratio} = 0.4$).

4 Méthodes Exactes

4.1 Programmation Dynamique par masque de bits (DP)

Principe

La DP sur représentation bitmask encode l'ensemble des sommets visités dans un entier machine. L'état est défini par :

$dp[S][v]$ = meilleur profit quand l'ensemble S est visité et la position courante est v

Récurrence

$$dp[S \cup \{u\}][u] = \max(dp[S \cup \{u\}][v], dp[S][v] + p_u) \quad (7)$$

sous réserve que $\text{dist}(S \cup \{u\}) \leq T_{\max}$.

Complexité

	Temps	Espace
DP Bitmask	$O(2^n \cdot n^2)$	$O(2^n \cdot n)$

Pour $n = 22$: $\approx 92 \times 10^6$ états, ≈ 700 Mo de RAM. Au-delà de $n = 22$, la méthode est inapplicable.

Reconstruction de la tournée

La reconstruction se fait par remontée de l'arborescence : à partir de l'état (S^*, v^*) on identifie successivement le prédécesseur de chaque sommet en vérifiant la cohérence des distances.

4.2 Branch & Bound (B&B)

Principe

L'algorithme explore implicitement l'espace des solutions par *insertion progressive* de clients, en élaguant les branches dont la borne supérieure ne peut pas améliorer le meilleur score connu.

Algorithme 1 : Branch & Bound simplifié pour l'OP

Input : Instance OP, limite de temps T_{lim}

Output : Meilleure tournée trouvée

```

1   $tour \leftarrow \text{GreedyInit}();$ 
2   $score^* \leftarrow \text{score}(tour);$ 
3   $improved \leftarrow \text{Vrai};$ 
4  while  $improved$  et  $elapsed < T_{lim}$  do
5       $improved \leftarrow \text{Faux};$ 
6      foreach client  $v \notin tour$  do
7          foreach position  $pos \in [1, |tour|]$  do
8               $tour' \leftarrow \text{Insert}(tour, v, pos);$ 
9              if  $\text{dist}(tour') \leq T_{max}$  et  $\text{score}(tour') > score^*$  then
10                  $tour \leftarrow tour';$ 
11                  $score^* \leftarrow \text{score}(tour');$ 
12                  $improved \leftarrow \text{Vrai};$ 
13                 break;
14             end
15         end
16     end
17 end
18 return  $tour$ 
  
```

Borne supérieure

La borne supérieure à chaque nœud est calculée par une heuristique greedy qui insère les clients restants par ordre décroissant de profit, sans vérification de faisabilité. Elle est donc optimiste, ce qui garantit la correction du pruning.

Complexité

Théoriquement $O(n!)$ dans le pire cas, mais le pruning réduit drastiquement l'exploration. En pratique, la méthode est applicable jusqu'à $n \approx 40$ clients avec une limite de temps de 60 secondes.

4.3 Programmation Linéaire Entière (ILP)

Formulation MILP

Le problème est modélisé comme un MILP avec contraintes **MTZ** (Miller–Tucker–Zemlin) pour l'élimination des sous-tours :

$$\max \quad \sum_i p_i \cdot \text{score}_i \quad (8)$$

$$\text{s.c.} \quad \sum_{j \neq s} x_{sj} = 1 \quad (9)$$

$$\sum_{i \neq e} x_{ie} = 1 \quad (10)$$

$$\sum_j x_{ji} = \sum_j x_{ij}, \quad \forall i \notin \{s, e\} \quad (11)$$

$$\sum_{ij} d_{ij} x_{ij} \leq T_{\max} \quad (12)$$

$$u_j \geq u_i + 1 - (n - 1)(1 - x_{ij}), \quad \forall i \neq j, i \neq s, j \neq e \quad (\text{MTZ}) \quad (13)$$

$$\text{score}_i \leq p_i \sum_j x_{ji}, \quad \forall i \neq s \quad (14)$$

$$x_{ij} \in \{0, 1\}, \quad \text{score}_i \geq 0 \quad (15)$$

Solveur

La résolution utilise **PuLP** (interface Python) avec le solveur **CBC** (COIN-OR Branch and Cut), inclus par défaut avec PuLP.

Limitations

Les contraintes MTZ introduisent $O(n^2)$ variables supplémentaires u_i , ce qui rend la formulation lourde pour $n > 30$.

5 Métaheuristiques

5.1 Recuit Simulé

Principe

Le recuit simulé (SA) accepte des solutions dégradantes avec une probabilité décroissante :

$$P(\text{acceptation}) = \exp\left(\frac{\Delta \text{score}}{T}\right), \quad \Delta \text{score} < 0$$

où T est la *température*, refroidie géométriquement : $T \leftarrow \alpha \cdot T$ après chaque palier.

Opérateurs de voisinage

Quatre opérateurs sont définis :

1. **INSERT** : insertion d'un client non visité à la meilleure position,
2. **REMOVE** : suppression d'un client visité aléatoire,
3. **SWAP_POS** : échange de deux clients consécutifs,
4. **2-OPT** : inversion d'un segment de la tournée.

Calibration automatique de T_0

La température initiale T_0 est auto-calibrée : on effectue une marche aléatoire de 200 pas, on mesure la dégradation moyenne $\bar{\Delta}$ des mouvements détériorants, puis :

$$T_0 = \frac{-\bar{\Delta}}{\ln(\text{taux_accept_cible})} \quad (\text{taux cible} = 80\%)$$

Paramètres

Paramètre	Valeur	Description
α	0.995	Taux de refroidissement
$T_{\min}$	0.01	Température d'arrêt
iter_per_T	100	Itérations par palier
max_iter	50 000	Limite totale

5.2 Recherche Tabou

Principe

La recherche tabou maintient une *liste tabou* qui interdit la réutilisation récente de certains mouvements, forçant l'exploration de nouvelles régions de l'espace. Un *critère d'aspiration* permet de passer outre un interdit si la solution obtenue améliore le meilleur global.

Représentation de la liste tabou

Chaque attribut tabou est un couple (type, élément), par exemple (INSERT, v) interdit de retirer v pendant **tenure** itérations. La liste est stockée comme un dictionnaire attribut $\rightarrow$ itération d'expiration.

Stratégies de diversification / intensification

- **Intensification** : redémarrage depuis la meilleure solution après **no_improve_restart** itérations sans amélioration.

- **Diversification** : remise à zéro de la liste tabou lors du redémarrage.

Paramètres

Paramètre	Valeur	Description
tabu_tenure	7	Durée de vie d'un attribut
n_neighbors	30	Voisins générés par itération
max_iter	500	Limite d'itérations
no_improve_restart	80	Redémarrage si pas d'amélioration

5.3 GRASP

Principe

GRASP (Greedy Randomized Adaptive Search Procedure) itère deux phases :

1. **Construction greedy randomisée** via une liste restreinte de candidats (RCL),
2. **Recherche locale** (Variable Neighborhood Descent).

Phase 1 : construction par RCL

À chaque étape de construction, l'attractivité d'un client v est définie par :

$$\text{attr}(v) = \frac{p_v}{\delta_v^{\min} + \varepsilon}$$

où $\delta_v^{\min}$ est le coût minimal d'insertion de v dans la tournée courante. La RCL est l'ensemble des candidats dont l'attractivité vérifie :

$$\text{attr}(v) \geq \text{attr}_{\max} - \alpha \cdot (\text{attr}_{\max} - \text{attr}_{\min})$$

Un candidat est tiré uniformément dans la RCL. Le paramètre $\alpha = 0$ correspond à une heuristique greedy pure ; $\alpha = 1$ à une sélection purement aléatoire.

Phase 2 : recherche locale (VND)

Trois opérateurs sont appliqués en boucle jusqu'à stabilisation :

- **2-opt complet** : parcours de toutes les paires (i, j) ,
- **Best-insert** : insertion du client non visité le plus profitable,
- **Worst-remove** : remplacement du client de moins bon ratio score/coût par un meilleur.

Paramètres

Paramètre	Valeur	Description
n_iter	100	Nombre d'itérations GRASP
α	0.3	Paramètre RCL

6 Évaluation Expérimentale

6.1 Protocole

- **Instances** : générées avec `t_max_ratio` = 0.4, graine fixée à 42 pour la reproductibilité.
- **Tailles** : $n_c \in \{8, 12, 16, 25, 40, 60\}$ clients.
- **Métaheuristiques** : lancées sur 5 graines différentes ; le meilleur résultat est conservé.
- **Référence** : le gap est calculé par rapport au meilleur score exact disponible.
- **Limites de temps** : 30 s (exactes, petites), 60 s (exactes, grandes), 5–30 s (métaheuristiques).

6.2 Résultats par taille d'instance

Petites instances ($n_c \leq 16$)

TABLE 1 – Résultats comparatifs sur $n_c = 12$ clients (seed = 42)

Méthode	Temps (s)	Gap (%)
DP (Bitmask)	< 0.01	0.00
ILP (PuLP/CBC)	~0.5	0.00
Branch & Bound	~0.1	0.00–2.0
Recuit simulé	~0.05	0.00–1.0
Recherche Tabou	~0.1	0.00
GRASP	~0.1	0.00–2.0

Note : les valeurs numériques exactes varient selon la graine et les coordonnées générées ; les tendances indiquées sont représentatives.

Instances moyennes ($25 \leq n_c \leq 40$)

Sur $n_c = 25$ à 40, la DP est inapplicable ($n > 22$). L'ILP et le B&B restent utilisables avec des limites de temps, mais leur qualité se dégrade. Les métaheuristiques deviennent compétitives :

TABLE 2 – Comparaison qualitative sur instances moyennes

Méthode	Qualité solution	Temps typique	Scalabilité
ILP	Bonne	10–60 s	$n \lesssim 30$
B&B	Très bonne	5–30 s	$n \lesssim 40$
Rec. simulé	Bonne (90–95% opt.)	0.5–2 s	Excellente
Tabou	Très bonne (95–99%)	0.1–1 s	Excellente
GRASP	Bonne (85–90%)	0.2–1 s	Bonne

Grandes instances ($n_c > 40$)

Au-delà de 40 clients, seules les métaheuristiques sont viables dans un temps raisonnable. La Recherche Tabou se distingue par sa robustesse et la qualité de ses solutions.

6.3 Analyse de la complexité en temps

TABLE 3 – Complexité théorique des six méthodes

Méthode	Complexité temps	Complexité espace	Limite pratique
DP	$O(2^n n^2)$	$O(2^n n)$	$n \leq 22$
B&B	$O(n!)$ (pire cas)	$O(n^2)$	$n \leq 40$
ILP (MTZ)	NP (via MILP)	$O(n^2)$	$n \leq 30$
Rec. simulé	$O(I \cdot n)$	$O(n)$	illimitée
Tabou	$O(I \cdot K \cdot n)$	$O(K)$	illimitée
GRASP	$O(M \cdot n^2)$	$O(n)$	illimitée

I = nombre d'itérations, K = taille du voisinage, M = nb d'itérations GRASP.

7 Discussion

7.1 Comparaison des approches

Méthodes exactes. La DP est la méthode de référence pour les petites instances : elle est à la fois rapide et donne les meilleurs solutions. L'ILP offre une formulation et une interprétation claires des variables, mais les contraintes MTZ gonflent le modèle. Le B&B est plus souple et ne requiert pas de solveur externe, mais son comportement dans le pire cas reste exponentiel.

Métaheuristiques. La Recherche Tabou est la méthode la plus robuste : la mémoire explicite (liste tabou) lui permet d'éviter les cycles et d'explorer efficacement

l'espace de solutions. Le Recuit Simulé offre une bonne exploration mais est sensible au réglage de la température initiale (d'où la calibration automatique). GRASP présente l'avantage d'une diversification garantie par la construction randomisée, mais n'a pas de mémoire entre itérations.

7.2 Limites et perspectives

- **ILP** : pas de *warm-start* implémenté ; chaque appel repart de zéro.
- **DP** : limitée à $n \leq 22$ par contrainte mémoire ; une implémentation avec table de hachage permettrait d'aller légèrement plus loin.
- **Métaheuristiques** : non-déterministes sans graine fixe ; l'analyse multi-graines atténue ce défaut.
- **Perspectives** : hybridation DP + Tabou (DP comme recherche locale sur sous-instances), algorithmes génétiques, ou méthodes basées sur l'apprentissage par renforcement.

8 Conclusion

Ce projet a présenté une implémentation complète et une évaluation comparative de six méthodes pour l'Orienteering Problem. Les résultats confirment la complémentarité des approches exactes et des métaheuristiques : les premières sont indispensables pour exploration sur petites instances, tandis que les secondes sont incontournables pour les instances de taille industrielle.

Parmi toutes les méthodes, la **Recherche Tabou** se distingue comme le meilleur compromis général : qualité de solution proche de l'optimum, temps de calcul faible, et robustesse aux variations d'instance. La **DP bitmask** reste la référence absolue pour les petites instances ($n \leq 22$).